Lab Sheet 10

SHOBHIT SHARMA
CU240250982
Roll No. 59

CampusHub: Full-Stack Application

Stack: Node.js, Express, MongoDB, Socket.io, Redis, React with Redux Toolkit, Jest/Vitest, Docker.

Project layout

```
campus-hub/
 server.js app.js auth.js cache.js sockets.js limiter.js validate.js asyncHandler.js bench.js
 models/      user.js event.js announcement.js
 routes/      events.js announcements.js
 tests/       api.test.js
 client/src/  store.js api.js authSlice.js eventSlice.js socket.js
              PrivateRoute.jsx EventList.jsx slices.test.js
 Dockerfile .dockerignore docker-compose.yml client/Dockerfile client/nginx.conf
```

Task 1: Authentication & Role-Based Access Control

Users register and log in with bcrypt-hashed passwords (cost 12). A short-lived access token (15 min) is sent in the Authorization header; a 7-day refresh token lives in an httpOnly cookie scoped to /api/auth. Every user has a tokenVersion, so logout revokes all refresh tokens issued earlier. Two middlewares guard routes: authenticate (valid token) and authorize (allowed roles). Registration always creates a STUDENT and rejects unknown body fields such as role, so nobody can self-promote to ADMIN.

setup

```
mkdir campus-hub && cd campus-hub
npm init -y
npm i express mongoose bcryptjs jsonwebtoken cookie-parser dotenv cors helmet compression \
 express-rate-limit zod socket.io ioredis
npm i -D jest supertest
```

asyncHandler.js

```
// forwards rejected promises to the central error handler (no try/catch in every route)
module.exports = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

models/user.js

```
const { Schema, model } = require('mongoose');
const userSchema = new Schema({
 fullName: { type: String, required: true, trim: true },
 email: { type: String, required: true, unique: true, lowercase: true, trim: true },
 password: { type: String, required: true, select: false },   // never returned by default
 role: { type: String, enum: ['ADMIN', 'STUDENT'], default: 'STUDENT' },
 tokenVersion: { type: Number, default: 0 },                    // bump to revoke refresh tokens
}, { timestamps: true });
userSchema.set('toJSON', { transform: (_d, o) => { delete o.password; delete o.__v; return o; } });
module.exports = model('User', userSchema);
```

auth.js

```
const router = require('express').Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('./models/user');
const wrap = require('./asyncHandler');
const { check, schemas } = require('./validate');

const signAccess = (u) => jwt.sign({ id: u.id, role: u.role }, process.env.ACCESS_KEY, { expiresIn: '15m' });
const signRefresh = (u) => jwt.sign({ id: u.id, tv: u.tokenVersion }, process.env.REFRESH_KEY, { expiresIn: '7d' });
const cookieOpts = {
 httpOnly: true, sameSite: 'strict', path: '/api/auth', maxAge: 7 * 864e5,
 secure: process.env.NODE_ENV === 'production',
};

// Middleware: must be logged in (expects "Authorization: Bearer <token>")
function authenticate(req, res, next) {
 const [scheme, token] = (req.headers.authorization || '').split(' ');
 if (scheme !== 'Bearer' || !token) return res.status(401).json({ error: 'Token missing' });
 try { req.user = jwt.verify(token, process.env.ACCESS_KEY); next(); }
 catch (e) { res.status(401).json({ error: e.name === 'TokenExpiredError' ? 'Token expired' : 'Token invalid' }); }
}
// Middleware: must hold one of the listed roles
const authorize = (...roles) => (req, res, next) =>
```

```
 roles.includes(req.user?.role) ? next() : res.status(403).json({ error: 'Access denied' });

router.post('/register', check(schemas.register), wrap(async (req, res) => {
 const { fullName, email, password } = req.body;
 if (await User.exists({ email })) return res.status(409).json({ error: 'Email already used' });
 const user = await User.create({ fullName, email, password: await bcrypt.hash(password, 12) });
 res.cookie('rt', signRefresh(user), cookieOpts);
 res.status(201).json({ accessToken: signAccess(user), user });
}));

router.post('/login', check(schemas.login), wrap(async (req, res) => {
 const user = await User.findOne({ email: req.body.email }).select('+password');
 if (!user || !(await bcrypt.compare(req.body.password, user.password)))
  return res.status(401).json({ error: 'Wrong email or password' });
 res.cookie('rt', signRefresh(user), cookieOpts);
 res.json({ accessToken: signAccess(user), user });
}));

router.post('/refresh', wrap(async (req, res) => {
 try {
  const { id, tv } = jwt.verify(req.cookies.rt, process.env.REFRESH_KEY);
  const user = await User.findById(id);
  if (!user || user.tokenVersion !== tv) throw new Error('revoked');
  res.json({ accessToken: signAccess(user) });
 } catch { res.status(401).json({ error: 'Please log in again' }); }
}));

router.post('/logout', wrap(async (req, res) => {
 try {
  const { id } = jwt.verify(req.cookies.rt, process.env.REFRESH_KEY);
  await User.findByIdAndUpdate(id, { $inc: { tokenVersion: 1 } });   // revoke old refresh tokens
 } catch {}
 res.clearCookie('rt', { path: '/api/auth' });
 res.sendStatus(204);
}));

router.get('/me', authenticate, wrap(async (req, res) => res.json(await User.findById(req.user.id))));

module.exports = { router, authenticate, authorize };
```

sample output

```
$ curl -s -X POST localhost:5000/api/auth/register -H "Content-Type: application/json" \
    -d '{"fullName":"Shobhit Sharma","email":"shobhit@coer.ac.in","password":"Secret@123"}'
{"accessToken":"eyJhbGciOiJIUzI1NiIs...","user":{"_id":"6710a1c4e2f3","fullName":"Shobhit Sharma",
 "email":"shobhit@coer.ac.in","role":"STUDENT","tokenVersion":0}}

$ curl -s localhost:5000/api/events
{"error":"Token missing"}

$ curl -s -X POST localhost:5000/api/events -H "Authorization: Bearer $STUDENT_TOKEN"
{"error":"Access denied"}
```

Task 2: Real-Time Notifications (Socket.io)

The Socket.io server checks the same JWT during the handshake and puts each socket in a room named after its role. When an admin posts an announcement, the server emits only to the STUDENT room, and admins receive a live count of connected students. The client reconnects automatically; if the token has expired it silently refreshes it first.

models/event.js

```
const { Schema, model } = require('mongoose');
module.exports = model('Event', new Schema({
 title: { type: String, required: true, trim: true },
 date: { type: Date, required: true, index: true },
 venue: String,
 rsvps: [{ type: Schema.Types.ObjectId, ref: 'User' }],
 createdBy: { type: Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true }));
```

models/announcement.js

```
const { Schema, model } = require('mongoose');
module.exports = model('Announcement', new Schema({
 title: { type: String, required: true, trim: true },
 message: { type: String, required: true },
 createdBy: { type: Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true }));
```

sockets.js

```
const { Server } = require('socket.io');
const jwt = require('jsonwebtoken');
module.exports = function attachSockets(httpServer, origin) {
```

```
 const io = new Server(httpServer, { cors: { origin, credentials: true } });

 // Reject connections that do not carry a valid access token
 io.use((socket, next) => {
  try {
   socket.user = jwt.verify(socket.handshake.auth?.token, process.env.ACCESS_KEY);
   next();
  } catch { next(new Error('unauthorized')); }
 });

 // tell every admin how many students are connected right now
 const pushOnline = async () =>
  io.to('ADMIN').emit('online', (await io.in('STUDENT').fetchSockets()).length);

 io.on('connection', (socket) => {
  socket.join(socket.user.role);                // 'ADMIN' or 'STUDENT' room
  pushOnline();
  socket.on('disconnect', (why) => {
   console.log('socket left:', socket.user.id, why);
   pushOnline();
  });
 });
 return io;
};
```

routes/announcements.js

```
const router = require('express').Router();
const Announcement = require('../models/announcement');
const wrap = require('../asyncHandler');
const { authenticate, authorize } = require('../auth');
const { check, schemas } = require('../validate');

router.use(authenticate);

router.get('/', wrap(async (req, res) => {
 res.json(await Announcement.find().sort({ createdAt: -1 }).limit(20).lean());
}));

router.post('/', authorize('ADMIN'), check(schemas.announcement), wrap(async (req, res) => {
 const item = await Announcement.create({ ...req.body, createdBy: req.user.id });
 req.app.get('io')?.to('STUDENT').emit('announcement', item);   // live push
 res.status(201).json(item);
}));

module.exports = router;
```

server.js

```
require('dotenv').config();
const http = require('http');
const mongoose = require('mongoose');
const app = require('./app');
const attachSockets = require('./sockets');

const PORT = process.env.PORT || 5000;
const server = http.createServer(app);
app.set('io', attachSockets(server, process.env.FRONTEND_ORIGIN || 'http://localhost:5173'));

mongoose.connect(process.env.MONGO_URI || 'mongodb://localhost:27017/campus')
 .then(() => server.listen(PORT, () => console.log(`API on :${PORT}`)))
 .catch((e) => { console.error(e); process.exit(1); });

// graceful shutdown (docker stop sends SIGTERM)
process.on('SIGTERM', () => server.close(() => mongoose.connection.close().then(() => process.exit(0))));
```

client/src/socket.js

```
import { io } from 'socket.io-client';
import { store } from './store';
import api from './api';
import { setToken, logout } from './authSlice';
import { newAnnouncement, setOnline } from './eventSlice';

let socket;
export function openSocket() {
 const token = store.getState().auth.token;
 if (!token || socket?.connected) return;
 socket = io(import.meta.env.VITE_API_ROOT || 'http://localhost:5000', {
  auth: { token },
  reconnection: true,              // auto-retry when the connection drops
  reconnectionDelayMax: 5000,
 });
 socket.on('announcement', (a) => { console.log('[live]', a.title); store.dispatch(newAnnouncement(a)); });
 socket.on('online', (n) => store.dispatch(setOnline(n)));
 // handshake rejected (expired token): get a fresh one, then connect again
 socket.on('connect_error', async (err) => {
  if (err.message !== 'unauthorized') return;
  try {
   const { data } = await api.post('/auth/refresh');
   store.dispatch(setToken(data.accessToken));
```

```
   socket.auth = { token: data.accessToken };
   socket.connect();
  } catch { store.dispatch(logout()); }
 });
}
export const closeSocket = () => { socket?.disconnect(); socket = undefined; };
```

sample output

```
server terminal                          student browser console
API on :5000                             [live] Mid-sem timetable released
socket left: 6710a1c4e2f3 transport close

admin terminal:
$ curl -s -X POST localhost:5000/api/announcements -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"title":"Mid-sem timetable released","message":"Check the portal."}'
{"title":"Mid-sem timetable released","message":"Check the portal.","_id":"6710b2d0aa41"}
```

Task 3: Redis Caching

GET /api/events is cached in Redis for 60 seconds, keyed by the full URL under a versioned prefix, so each page/search is cached separately. Only 200 responses are stored. The X-Cache header shows HIT, MISS or BYPASS (Redis down). Any create, delete or RSVP clears the cached lists using SCAN + UNLINK, which does not block Redis the way KEYS does. If Redis is down, requests fall back to MongoDB.

cache.js

```
const Redis = require('ioredis');
const redis = new Redis(process.env.REDIS_URL || 'redis://localhost:6379', {
 enableOfflineQueue: false, maxRetriesPerRequest: 1,
});
redis.on('error', (e) => console.log('redis:', e.message));

const PREFIX = 'events:v1:';
const TTL = 60; // seconds

async function cacheEvents(req, res, next) {
 const key = PREFIX + req.originalUrl;
 try {
  const hit = await redis.get(key);
  if (hit) { res.set('X-Cache', 'HIT'); return res.json(JSON.parse(hit)); }
 } catch { res.set('X-Cache', 'BYPASS'); return next(); }   // Redis down -> go to DB
 res.set('X-Cache', 'MISS');
 const send = res.json.bind(res);
 res.json = (body) => {
  if (res.statusCode === 200) redis.set(key, JSON.stringify(body), 'EX', TTL).catch(() => {});
  return send(body);
 };
 next();
}

async function clearEventCache() {
 try {
  for await (const keys of redis.scanStream({ match: PREFIX + '*', count: 100 }))
   if (keys.length) await redis.unlink(keys);
 } catch {}
}
module.exports = { cacheEvents, clearEventCache, redis };
```

routes/events.js

```
const router = require('express').Router();
const mongoose = require('mongoose');
const Event = require('../models/event');
const wrap = require('../asyncHandler');
const { authenticate, authorize } = require('../auth');
const { check, schemas } = require('../validate');
const { cacheEvents, clearEventCache } = require('../cache');

router.use(authenticate);
const badId = (req, res) => !mongoose.isValidObjectId(req.params.id) && res.status(400).json({ error: 'Bad id' });

// List with pagination + title search (cached)
router.get('/', cacheEvents, wrap(async (req, res) => {
 const page = Math.max(parseInt(req.query.page) || 1, 1);
 const limit = Math.min(parseInt(req.query.limit) || 5, 50);
 const search = String(req.query.search || '').trim().slice(0, 60);
 const filter = search ? { title: new RegExp(search.replace(/[.*+?^${}()|[\]\\]/g, '\\$&'), 'i') } : {};
 const [items, total] = await Promise.all([
  Event.find(filter).sort({ date: 1 }).skip((page - 1) * limit).limit(limit).lean(),
  Event.countDocuments(filter),
 ]);
 res.json({ items, page, totalPages: Math.max(Math.ceil(total / limit), 1), total });
}));
```

```
router.post('/', authorize('ADMIN'), check(schemas.event), wrap(async (req, res) => {
 const ev = await Event.create({ ...req.body, createdBy: req.user.id });
 await clearEventCache();
 res.status(201).json(ev);
}));

router.post('/:id/rsvp', authorize('STUDENT'), wrap(async (req, res) => {
 if (badId(req, res)) return;
 const ev = await Event.findByIdAndUpdate(req.params.id,
  { $addToSet: { rsvps: req.user.id } }, { new: true });
 if (!ev) return res.status(404).json({ error: 'Event not found' });
 await clearEventCache();
 res.json(ev);
}));

router.delete('/:id', authorize('ADMIN'), wrap(async (req, res) => {
 if (badId(req, res)) return;
 await Event.findByIdAndDelete(req.params.id);
 await clearEventCache();
 res.sendStatus(204);
}));

module.exports = router;
```

bench.js

```
// node bench.js <jwt> [runs]  -> compares the first call with repeated (cached) calls
const [token, runsArg] = process.argv.slice(2);
const N = Number(runsArg) || 100;
const url = 'http://localhost:5000/api/events?page=1';

async function timeOne() {
 const t = performance.now();
 const r = await fetch(url, { headers: { Authorization: 'Bearer ' + token } });
 await r.json();
 return { ms: performance.now() - t, cache: r.headers.get('x-cache') };
}

(async () => {
 const first = await timeOne();
 const runs = [];
 for (let i = 0; i < N; i++) runs.push(await timeOne());
 const ms = runs.map((r) => r.ms).sort((a, b) => a - b);
 const avg = ms.reduce((a, b) => a + b) / N;
 const hits = runs.filter((r) => r.cache === 'HIT').length;
 console.log(`1st call   : ${first.ms.toFixed(2)} ms [${first.cache}]`);
 const p95 = ms[Math.ceil(N * 0.95) - 1];
 console.log(`cached x${N}: avg ${avg.toFixed(2)} ms | p95 ${p95.toFixed(2)} ms | hits ${hits}/${N}`);
})();
```

sample output

```
$ node bench.js $TOKEN 100
1st call   : 18.42 ms [MISS]
cached x100: avg 2.31 ms | p95 3.74 ms | hits 100/100

$ curl -si localhost:5000/api/events?page=1 -H "Authorization: Bearer $T" | grep -i x-cache
X-Cache: HIT
(after admin creates an event)  ->  X-Cache: MISS
(redis container stopped)       ->  X-Cache: BYPASS   (data still served from MongoDB)
```

Task 4: Frontend with State Management (React + Redux Toolkit)

Redux Toolkit holds auth and events state, each with loading and error status. The access token is kept in memory only (never localStorage). An axios interceptor attaches the token and, on a 401, asks /auth/refresh for a new one and retries once; several requests failing together share a single refresh call. Search is debounced by 300 ms and protected pages use PrivateRoute.

client/src/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import auth from './authSlice';
import events from './eventSlice';

export const store = configureStore({ reducer: { auth, events } });
```

client/src/api.js

```
import axios from 'axios';
import { store } from './store';
import { setToken, logout } from './authSlice';

const api = axios.create({
 baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
 withCredentials: true,           // sends the refresh cookie
```

```
});

api.interceptors.request.use((cfg) => {
 const t = store.getState().auth.token;
 if (t) cfg.headers.Authorization = `Bearer ${t}`;
 return cfg;
});

let refreshing = null;            // shared promise: one refresh call for many failed requests
api.interceptors.response.use(null, async (err) => {
 const req = err.config;
 if (err.response?.status === 401 && !req._tried && !req.url.includes('/auth/')) {
  req._tried = true;
  try {
   refreshing = refreshing || api.post('/auth/refresh').finally(() => { refreshing = null; });
   const { data } = await refreshing;
   store.dispatch(setToken(data.accessToken));
   return api(req);
  } catch { store.dispatch(logout()); }
 }
 return Promise.reject(err);
});

export default api;
```

client/src/authSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import api from './api';

export const loginUser = createAsyncThunk('auth/login', async (creds, { rejectWithValue }) => {
 try { return (await api.post('/auth/login', creds)).data; }
 catch (e) { return rejectWithValue(e.response?.data?.error || 'Login failed'); }
});
export const logoutUser = createAsyncThunk('auth/logout', async () => {
 try { await api.post('/auth/logout'); } catch {}      // server revokes refresh tokens
});

const initialState = { user: null, token: null, status: 'idle', error: null };

const slice = createSlice({
 name: 'auth',
 initialState,
 reducers: {
  setToken: (s, a) => { s.token = a.payload; },
  logout: () => initialState,
 },
 extraReducers: (b) => {
  b.addCase(loginUser.pending, (s) => { s.status = 'loading'; s.error = null; })
   .addCase(loginUser.fulfilled, (s, a) => {
    s.status = 'ready'; s.user = a.payload.user; s.token = a.payload.accessToken;
   })
   .addCase(loginUser.rejected, (s, a) => { s.status = 'failed'; s.error = a.payload; })
   .addCase(logoutUser.fulfilled, () => initialState);
 },
});

export const selectRole = (s) => s.auth.user?.role;
export const { setToken, logout } = slice.actions;
export default slice.reducer;
```

client/src/eventSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import api from './api';

export const loadEvents = createAsyncThunk('events/load', async ({ page = 1, search = '' } = {}) =>
 (await api.get('/events', { params: { page, search } })).data);
export const rsvp = createAsyncThunk('events/rsvp', async (id) =>
 (await api.post(`/events/${id}/rsvp`)).data);

const slice = createSlice({
 name: 'events',
 initialState: { items: [], page: 1, totalPages: 1, unread: 0, online: 0, status: 'idle' },
 reducers: {
  newAnnouncement: (s) => { s.unread += 1; },
  clearUnread: (s) => { s.unread = 0; },
  setOnline: (s, a) => { s.online = a.payload; },
 },
 extraReducers: (b) => {
  b.addCase(loadEvents.pending, (s) => { s.status = 'loading'; })
   .addCase(loadEvents.fulfilled, (s, { payload }) => {
    s.status = 'ready'; s.items = payload.items; s.page = payload.page; s.totalPages = payload.totalPages;
   })
   .addCase(loadEvents.rejected, (s) => { s.status = 'failed'; })
   .addCase(rsvp.fulfilled, (s, { payload }) => {
    const i = s.items.findIndex((e) => e._id === payload._id);
    if (i >= 0) s.items[i] = payload;
   });
 },
});
```

```
export const { newAnnouncement, clearUnread, setOnline } = slice.actions;
export default slice.reducer;
```

client/src/PrivateRoute.jsx

```
import { useSelector } from 'react-redux';
import { Navigate, Outlet } from 'react-router-dom';

export default function PrivateRoute({ roles }) {
 const { token, user } = useSelector((s) => s.auth);
 if (!token) return <Navigate to="/login" replace />;
 if (roles && !roles.includes(user?.role)) return <Navigate to="/" replace />;
 return <Outlet />;
}
```

client/src/EventList.jsx

```
import { useEffect, useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { loadEvents, rsvp } from './eventSlice';

export default function EventList() {
 const dispatch = useDispatch();
 const { items, page, totalPages, status } = useSelector((s) => s.events);
 const user = useSelector((s) => s.auth.user);
 const [search, setSearch] = useState('');

 // debounce: wait 300 ms after the last keystroke before calling the API
 useEffect(() => {
  const t = setTimeout(() => dispatch(loadEvents({ page: 1, search })), 300);
  return () => clearTimeout(t);
 }, [search, dispatch]);

 return (
  <section>
   <h2>Campus Events</h2>
   <input placeholder="Search by title" value={search} onChange={(e) => setSearch(e.target.value)} />
   {status === 'loading' && <p>Loading...</p>}
   {status === 'ready' && items.length === 0 && <p>No events found.</p>}
   <ul>
    {items.map((e) => {
     const going = e.rsvps?.includes(user?.id || user?._id);
     return (
      <li key={e._id}>
       {e.title} - {new Date(e.date).toLocaleDateString()} ({e.rsvps?.length || 0} going)
       {user?.role === 'STUDENT' && (
        <button disabled={going} onClick={() => dispatch(rsvp(e._id))}>{going ? 'Going' : 'RSVP'}</button>
       )}
      </li>
     );
    })}
   </ul>
   <button disabled={page <= 1} onClick={() => dispatch(loadEvents({ page: page - 1, search }))}>Prev</button>
   <span> Page {page} of {totalPages} </span>
   <button disabled={page >= totalPages} onClick={() => dispatch(loadEvents({ page: page + 1, search }))}>Next</button>
  </section>
 );
}
```

sample output

```
Network tab (access token expired, two requests fired together)
GET  /api/events?page=1        401
GET  /api/announcements        401
POST /api/auth/refresh         200      <- only ONE refresh call
GET  /api/events?page=1        200      (retried once)
GET  /api/announcements        200      (retried once)
```

Task 5: Security Hardening

helmet sets secure HTTP headers, CORS is limited to the frontend origin, failed logins are limited to 5 per 15 minutes per IP (successful logins are not counted), registration is limited to 10 per hour, request bodies are validated with zod, unknown routes return a JSON 404 and errors never expose stack traces.

limiter.js

```
const rateLimit = require('express-rate-limit');
const make = (windowMs, max, error, extra = {}) =>
 rateLimit({ windowMs, max, standardHeaders: true, legacyHeaders: false, message: { error }, ...extra });

exports.loginLimiter = make(15 * 60 * 1000, 5, 'Too many attempts, try again in 15 minutes',
 { skipSuccessfulRequests: true });                    // only failed logins use up the quota
exports.registerLimiter = make(60 * 60 * 1000, 10, 'Too many sign-ups from this IP, try later');
```

validate.js

```
const { z } = require('zod');

const email = z.string().trim().toLowerCase().email();
const title = z.string().trim().min(3).max(120);

exports.schemas = {
 register: z.object({
  fullName: z.string().trim().min(2).max(80),
  email,
  password: z.string().min(8).max(72),          // bcrypt only reads the first 72 bytes
 }).strict(),                                    // unknown keys (e.g. role) -> 400
 login: z.object({ email, password: z.string().min(1) }),
 event: z.object({ title, date: z.coerce.date(), venue: z.string().trim().max(120).optional() }),
 announcement: z.object({ title, message: z.string().trim().min(1).max(1000) }),
};

// usage: router.post('/login', check(schemas.login), ...)
exports.check = (schema) => (req, res, next) => {
 const r = schema.safeParse(req.body);
 if (!r.success) return res.status(400).json({
  error: 'Invalid input',
  details: r.error.issues.map((i) => `${i.path.join('.') || 'body'}: ${i.message}`),
 });
 req.body = r.data;                              // keeps only validated fields
 next();
};
```

app.js

```
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const compression = require('compression');
const cookieParser = require('cookie-parser');
const { router: authRouter } = require('./auth');
const { loginLimiter, registerLimiter } = require('./limiter');

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_ORIGIN || 'http://localhost:5173', credentials: true }));
app.use(compression());
app.use(express.json({ limit: '10kb' }));
app.use(cookieParser());

app.get('/api/health', (req, res) => res.json({ ok: true }));
app.use('/api/auth/login', loginLimiter);
app.use('/api/auth/register', registerLimiter);
app.use('/api/auth', authRouter);
app.use('/api/events', require('./routes/events'));
app.use('/api/announcements', require('./routes/announcements'));

app.use((req, res) => res.status(404).json({ error: 'Route not found' }));

// Central error handler: no stack traces sent to the client
app.use((err, req, res, next) => {
 console.error(err.message);
 const bad = err.type === 'entity.parse.failed' || err.type === 'entity.too.large';
 res.status(bad ? 400 : 500).json({ error: bad ? 'Bad request body' : 'Something went wrong' });
});

module.exports = app;
```

sample output

```
$ curl -s -X POST localhost:5000/api/auth/register -H "Content-Type: application/json" \
    -d '{"fullName":"Eve","email":"eve@x.com","password":"Secret@123","role":"ADMIN"}'
{"error":"Invalid input","details":["body: Unrecognized key(s) in object: 'role'"]}

$ (6th wrong password within 15 minutes)
HTTP/1.1 429 Too Many Requests
{"error":"Too many attempts, try again in 15 minutes"}

$ curl -s localhost:5000/api/nothing
{"error":"Route not found"}

$ curl -sI localhost:5000/api/health | grep -iE "x-content-type|strict-transport|x-powered"
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=15552000; includeSubDomains
```

Task 6: Testing (Jest + Supertest, Vitest)

Backend tests run against the real Express app with the Redis cache mocked, and only on routes that fail before touching the database (missing or expired token, wrong role, bad input), so neither MongoDB nor Redis is needed. Frontend tests check the Redux reducers.

tests/api.test.js

```
process.env.ACCESS_KEY = 'test-access';
process.env.REFRESH_KEY = 'test-refresh';
jest.mock('../cache', () => ({ cacheEvents: (_q, _s, next) => next(), clearEventCache: async () => {} }));

const request = require('supertest');
const jwt = require('jsonwebtoken');
const app = require('../app');

const tokenFor = (role, opts = {}) => jwt.sign({ id: 'u1', role }, process.env.ACCESS_KEY, opts);
const as = (role) => 'Bearer ' + tokenFor(role);

describe('authentication', () => {
 test('401 without a token', async () => {
  expect((await request(app).get('/api/events')).status).toBe(401);
 });
 test('401 with a garbage token', async () => {
  const res = await request(app).get('/api/events').set('Authorization', 'Bearer abc');
  expect(res.status).toBe(401);
 });
 test('401 with an expired token', async () => {
  const res = await request(app).get('/api/events')
   .set('Authorization', 'Bearer ' + tokenFor('STUDENT', { expiresIn: -10 }));
  expect(res.body.error).toBe('Token expired');
 });
});

describe('authorization', () => {
 test.each([
  ['post', '/api/events', 'STUDENT'],
  ['post', '/api/announcements', 'STUDENT'],
  ['post', '/api/events/abc/rsvp', 'ADMIN'],
  ['delete', '/api/events/abc', 'STUDENT'],
 ])('403: %s %s as %s', async (method, url, role) => {
  const res = await request(app)[method](url).set('Authorization', as(role)).send({ title: 'Fest' });
  expect(res.status).toBe(403);
 });
});

describe('input validation', () => {
 const reg = { fullName: 'Test User', email: 'a@b.com', password: 'longenough1' };
 test('400 for a short password on register', async () => {
  expect((await request(app).post('/api/auth/register').send({ ...reg, password: '123' })).status).toBe(400);
 });
 test('400 when register tries to set a role', async () => {
  expect((await request(app).post('/api/auth/register').send({ ...reg, role: 'ADMIN' })).status).toBe(400);
 });
 test('400 for an invalid email on login', async () => {
  expect((await request(app).post('/api/auth/login').send({ email: 'nope', password: 'x' })).status).toBe(400);
 });
 test('400 when an admin sends a too-short event title', async () => {
  const res = await request(app).post('/api/events').set('Authorization', as('ADMIN'))
   .send({ title: 'ab', date: '2026-11-01' });
  expect(res.status).toBe(400);
 });
});

describe('misc', () => {
 test('health check works', async () => {
  expect((await request(app).get('/api/health')).body.ok).toBe(true);
 });
 test('unknown route gives a JSON 404', async () => {
  const res = await request(app).get('/api/nope');
  expect([res.status, res.body.error]).toEqual([404, 'Route not found']);
 });
 test('helmet headers are present', async () => {
  expect((await request(app).get('/api/health')).headers['x-content-type-options']).toBe('nosniff');
 });
});
```

client/src/slices.test.js

```
import { describe, it, expect } from 'vitest';
import reducer, { newAnnouncement, clearUnread, setOnline } from './eventSlice';
import auth, { setToken, logout, loginUser } from './authSlice';

describe('eventSlice', () => {
 it('counts unread announcements', () => {
  let s = reducer(undefined, newAnnouncement());
  s = reducer(s, newAnnouncement());
  expect(s.unread).toBe(2);
 });
 it('clears the badge', () => {
  const s = reducer({ items: [], page: 1, totalPages: 1, unread: 3 }, clearUnread());
  expect(s.unread).toBe(0);
 });
 it('stores the online student count', () => {
  expect(reducer(undefined, setOnline(7)).online).toBe(7);
 });
});
```

```
describe('authSlice', () => {
 it('stores and clears the token', () => {
  let s = auth(undefined, setToken('abc'));
  expect(s.token).toBe('abc');
  s = auth(s, logout());
  expect(s.token).toBeNull();
 });
 it('saves user and token after login', () => {
  const s = auth(undefined, loginUser.fulfilled({ user: { role: 'ADMIN' }, accessToken: 't1' }, 'id', {}));
  expect([s.user.role, s.token, s.status]).toEqual(['ADMIN', 't1', 'ready']);
 });
 it('keeps the error message when login fails', () => {
  const s = auth(undefined, loginUser.rejected(null, 'id', {}, 'Wrong email or password'));
  expect([s.status, s.error]).toEqual(['failed', 'Wrong email or password']);
 });
});
```

run

```
# backend (package.json: "test": "jest --runInBand --verbose")
npm test
# frontend
cd client && npx vitest run
```

sample output

```
 PASS  tests/api.test.js
  authentication
    ✓ 401 without a token (38 ms)
    ✓ 401 with a garbage token (6 ms)
    ✓ 401 with an expired token (5 ms)
  authorization
    ✓ 403: post /api/events as STUDENT (4 ms)
    ✓ 403: post /api/announcements as STUDENT (3 ms)
    ✓ 403: post /api/events/abc/rsvp as ADMIN (3 ms)
    ✓ 403: delete /api/events/abc as STUDENT (3 ms)
  input validation
    ✓ 400 for a short password on register (5 ms)
    ✓ 400 when register tries to set a role (4 ms)
    ✓ 400 for an invalid email on login (3 ms)
    ✓ 400 when an admin sends a too-short event title (4 ms)
  misc
    ✓ health check works (3 ms)
    ✓ unknown route gives a JSON 404 (3 ms)
    ✓ helmet headers are present (3 ms)

Tests:       14 passed, 14 total

 ✓ src/slices.test.js (6 tests) 7ms
 Test Files  1 passed (1)
      Tests  6 passed (6)
```

Task 7: Containerisation & Deployment (Docker)

Each service runs in its own container. The backend uses a small Node image, runs as a non-root user and has a health check; the frontend is built with Node and served by Nginx with gzip and asset caching (multi-stage build). docker compose starts MongoDB, Redis, the API and the web app together on one network, and each service waits until the ones it needs are healthy.

Dockerfile (backend)

```
FROM node:20-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --chown=node:node . .
USER node
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=3s --retries=3 CMD wget -qO- http://localhost:5000/api/health || exit 1
CMD ["node", "server.js"]
```

.dockerignore

```
node_modules
client
tests
.env
.git
```

client/Dockerfile

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
```

```
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

client/nginx.conf

```
server {
 listen 80;
 root /usr/share/nginx/html;
 gzip on;
 gzip_types text/css application/javascript application/json image/svg+xml;
 location /assets/ { expires 7d; add_header Cache-Control "public, immutable"; }
 location / { try_files $uri /index.html; }   # lets React Router handle paths
}
```

docker-compose.yml

```
services:
 mongo:
  image: mongo:7
  volumes: [mongo-data:/data/db]
  healthcheck:
   test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
   interval: 10s
   timeout: 5s
   retries: 5
 redis:
  image: redis:7-alpine
  healthcheck:
   test: ["CMD", "redis-cli", "ping"]
   interval: 10s
   timeout: 3s
   retries: 5
 api:
  build: .
  restart: unless-stopped
  depends_on:
   mongo: { condition: service_healthy }
   redis: { condition: service_healthy }
  environment:
   PORT: 5000
   MONGO_URI: mongodb://mongo:27017/campus
   REDIS_URL: redis://redis:6379
   ACCESS_KEY: ${ACCESS_KEY}
   REFRESH_KEY: ${REFRESH_KEY}
   FRONTEND_ORIGIN: http://localhost:3000
  ports: ["5000:5000"]
 web:
  build: ./client
  restart: unless-stopped
  depends_on:
   api: { condition: service_healthy }
  ports: ["3000:80"]
volumes:
 mongo-data:
```

.env (next to docker-compose.yml, do not commit)

```
ACCESS_KEY=put-a-long-random-string-here
REFRESH_KEY=put-another-long-random-string-here
```

run

```
docker compose up --build -d
docker compose ps
```

sample output

```
NAME                SERVICE   STATUS                    PORTS
campus-hub-mongo-1   mongo     Up 41 seconds (healthy)   27017/tcp
campus-hub-redis-1   redis     Up 41 seconds (healthy)   6379/tcp
campus-hub-api-1     api       Up 26 seconds (healthy)   0.0.0.0:5000->5000/tcp
campus-hub-web-1     web       Up 11 seconds             0.0.0.0:3000->80/tcp

$ curl -s localhost:5000/api/health
{"ok":true}
```